COMP-308
Fall 2024

COMP-308
Fall 2024

Generative AI and Recent Advancements in AI

Generative AI and Recent Advancements in AI

- Summarize Recurrent Neural Networks (RNNs) and Long Short term Memory (LSTM) networks
- Summarize New Models and Architectures.
- Discuss Current Advancements in Generative AI

Recurrent **N**eural **N**etworks - RNNs

- In many applications such as those with **sequential data** (stock prices, ECG readings, etc.), neural networks **need memory**.
  - In modeling natural language **context is important**:
    - the meaning of a word late in a sentence is affected by the meaning of words earlier in the sentence:

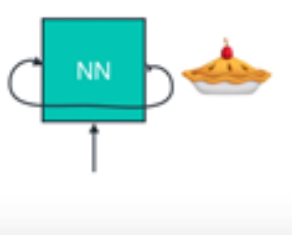
      I lived ten years in Spain

      I can speak fluent _____

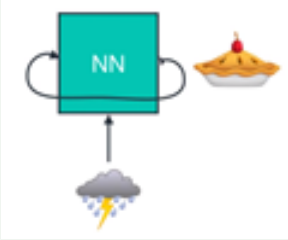
  - Language models **need the information to persist**.
- **R**ecurrent **N**eural **N**etworks (RNNs) solve this issue.

RNNs

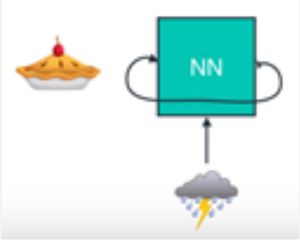
- A Recurrent Neural Network has **two inputs**, the **present** and the **recent past**.
- This allows the sequence of data to hold information about what is coming next.
- A Feed-Forward Neural Network assigns a weight matrix to its inputs and then produces the output.
- Note that **RNN's apply weights to the current and also to the previous input**.



Previous output



Current input

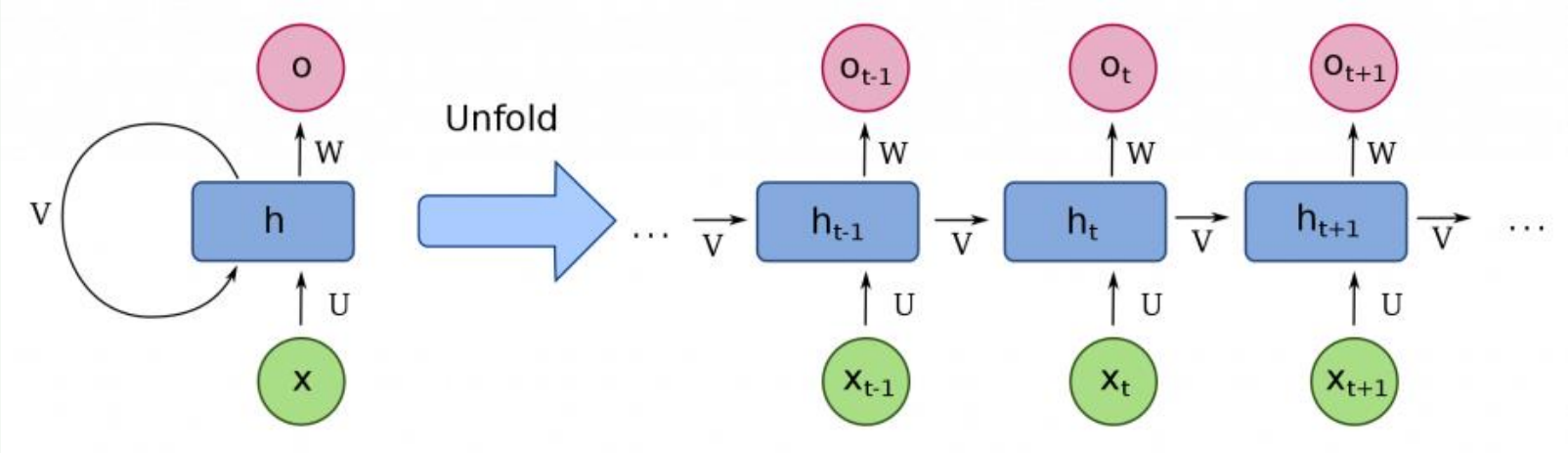


Previous and Current input

https://www.youtube.com/watch?v=UNmqTiOnRfg&t=3s

RNNs

- The loop labeled **V** represents the **feedback operation of the neuron**.
- As **future input values** (**X**) are given to the neuron, the **output** (**v**) from the previous activation reaches the input and becomes an input factor.
- As the graphic illustrates, this can be modeled as a linear series of simple neurons.



- By passing the output of the previous activation in sequence to the next, **RNN can remember** key values when it's learning.

Long short-term memory (LSTM)

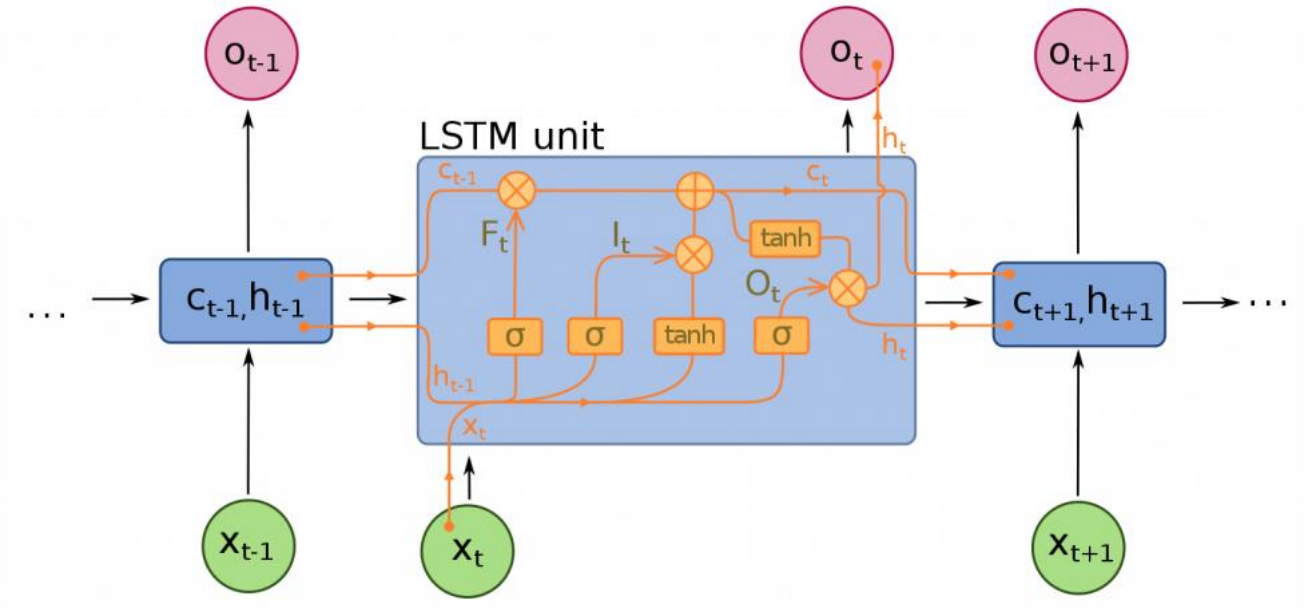
- An LSTM network is a special type of RNN.
- The LSTM network gets its name due to the fact that it **can retain short-term memory** (for example, memory of a word used earlier in a sentence) **for long periods of time**:
  - I lived ten years in Spain
  - I can speak fluent _____
- RNN may find it challenging to predict the word "Spanish" after the sentence "I lived ten years in Spain" due to vanishing gradient problems.
- LSTMs address this by utilizing two state vectors at each time step:
  - The **hidden state h(t)**, which is similar to the hidden state in RNNs and is used for predictions and output.
  - The **cell state** C(t) which serves as the network's long-term memory, efficiently stores and accesses information over time.

LSTM

- A particular structure, called **gates**, allows you to **remove** (*forget*) or **add** (*remember*) information to the cell state C(t) at each time step based on the input values x(t) and hidden state h(t−1).
- Specifically, the LSTM neuron has a **cell** and **three *gates*** it uses internally:
  - The ***cell*** remembers values over arbitrary time intervals.
  - ***input gate*** - Determines the extent to which new information from the current input x(t) is added to the cell state.
  - ***forget gate*** - manages the memory of the neuron, ecides what portions of the cell state C(t−1) are no longer needed and can be removed.
  - ***output gate*** - Controls the amount of memory from the cell state that will be used to compute the current hidden state h(t) and the output.

LSTM

- Consider the following graphic by Francois Deloche:



- The **It** signal controls the **proportion of the input signal** that is allowed into the cell.
- The **Ot** signal controls **how much of the output is allowed out of the cell**.
- The **Ft** signal controls **how much of the previous value is retained** by the cell.
- These are all vector quantities, so that the **input**, **output**, and **memory** can be controlled on a **per-element basis**.

LSTM

- When a new piece of data (like a word in a sentence) comes in, the LSTM decides:
  - what to remember from the past (**forget** gate)
  - what new information is relevant (**input** gate)
  - what to output based on the current **input** and past information (output gate)
- The **cell state** acts as a kind of long-term memory, remembering important information over many steps
- The **hidden state** serves as **short-term memory**, capturing recent patterns.

LSTM

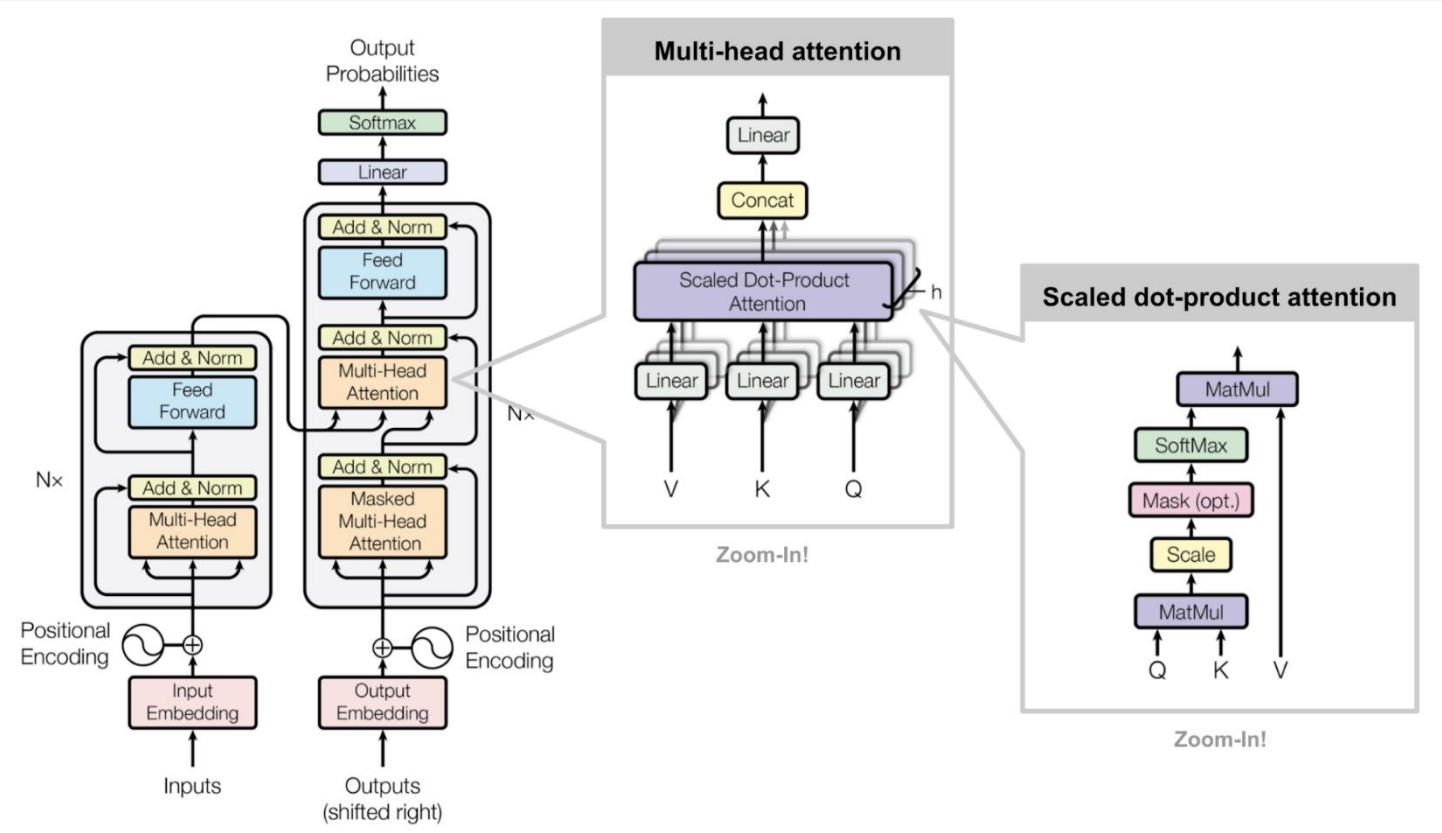
- The LSTM excels at **tasks that require memory and knowledge of prior values**, though the sophisticated inner workings of the cell (there are **five distinct activation functions** involved) lead to much longer training times.
- There are many state-of-the-art uses for LSTM networks.
- They are very popular in **audio analysis**, such as **speech recognition**, as audio is heavily time-dependent.
- Another state-of-the-art use case for LSTM is the CNN-LSTM, used for detection in a video clip.

Moving Toward Transformers

- Why Evolve Beyond LSTMs?
  - LSTMs can be slow and less effective with very long sequences.
- Transformers: Utilize attention mechanisms to process data more efficiently.
  - Better at handling long-range dependencies and large datasets.

Transformer Model

- Vaswani, et al., 2017, proposed "**transformer**" model, entirely built on the **self-attention mechanisms** without using sequence-aligned recurrent architecture:



https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Self-Attention Mechanism

- Allows the model to **weigh the importance of each token in the input sequence** based on its relation to all other tokens in the sequence.
- Computes the **attention weights based on the similarity** between each input element and all other elements.
- This allows the model to **capture long-range dependencies in the input sequence without the need for recurrent connections** or maintaining a hidden state.
- Can be seen as a type of **sequential modeling capability** that is similar to RNNs, but **more efficient and scalable for long sequences**.

Simplified Example Using the Task of Translating a Sentence from English to French

- English Sentence: "The cat sat on the mat."
- French Translation: "Le chat s'est assis sur le tapis.“
- **Step 1: Embedding** (each word is converted into high-dimensional vector)

  Input sentence: "The cat sat on the mat."

  Word embeddings (simplified):

  The - [1,0,0]

  cat - [0,1,0]

  sat - [0,0,1]

  on - [0,0,0.5]

  mat - [0.5,0,0]
- Vectors are learned from a large corpus of text using techniques like **word2vec**, **contextualized word embeddings**, etc.
- Each element in the vector, represents a “property” of the word, higher values suggest a stronger association with the property, while lower values might indicate a weaker connection or even an opposition.

Simplified Example Using the Task of Translating a Sentence from English to French

- **Step 2: Positional Encoding** (to keep track of the order of words)
  - Each word is assigned a **word embedding** (e.g., [1, 0, 0] for "The").
  - A small, high-dimensional vector is added to the embedding for each word, containing its **positional encoding** (e.g., [0.1, 0, 0] for "The").
  - The specific values in the positional encoding vector are calculated using mathematical functions to **capture relative positions** (not absolute positions like "first word").
  - **Words closer in sequence will have more similar positional encodings** compared to distant words.

Word	Word Embedding	Positional Encoding	Combined Representation
The	[1, 0, 0]	[0.1, 0, 0]	[1.1, 0, 0]
cat	[0, 1, 0]	[0, 0.1, 0]	[0, 1.1, 0]
sat	[0, 0, 1]	[0, 0, 0.1]	[0, 0, 1.1]

Simplified Example Using the Task of Translating a Sentence from English to French

- **Step 3: Attention Weights Calculation**

  Query for "sat" (Q): [0,0,1.1] – represents the word we are focusing on

  Keys for all words (K): [[1.1,0,0], [0,1.1,0], [0,0,1.1], [0,0,0.5], [0.5,0,0]] – represents all the words in the sentence.

  Dot products: Q * K = [0, 0, 1.21, 0.55, 0] - a measure of how well the keys (surrounding words) match the query word ("sat") meaning-wise.

  Scaled (assuming scale factor is 0.1) to improve the stability of the calculations and **softmax** applied to transform the scores into a probability distribution, ensuring that the sum of all attention scores for all words in the sentence equals 1.:

  e.g., Softmax([0, 0, 0.121, 0.055, 0]) => [0.2, 0.2, 0.28, 0.24, 0.2]

  - These simplified softmax calculations, represent the **weights assigned to each word in the sentence based on their relevance to the query word "sat".**
  - A higher score indicates a stronger semantic relationship between the word and the query. In this example, "sat" likely has a stronger relationship with "the" and "cat" compared to "on" and "mat".
  - These **weights are crucial for the model to focus its attention on the most relevant parts** of the sentence when processing the query word

Simplified Example Using the Task of Translating a Sentence from English to French

- **Step 4: Value Weighting**

  Values (V) (simplified and same as K for this example):

  V = K - represent the semantic information of each word.

  Weighted values: Attention weights * V (element-wise)

  => [0.2, 0.2, 0.28, 0.24, 0.2] * [[1.1,0,0], [0,1.1,0], [0,0,1.1], [0,0,0.5], [0.5,0,0]]

  ⇒ [[0.22,0,0], [0,0.22,0], [0,0,0.308], [0,0,0.12], [0.1,0,0]]

  - This process essentially scales the value of each word based on its relevance to the query ("sat"). Words with higher attention weights will have their semantic information weighted more heavily in the final context vector.

- **Step 5: Summation** (weighted values vectors element wise)

  Context vector for "sat":

  => [0.22 + 0 + 0 + 0 + 0.1, 0 + 0.22 + 0 + 0 + 0, 0 + 0 + 0.308 + 0.12 + 0] => [0.32, 0.22, 0.428]

- This vector incorporates information from "the" (0.32), "cat" (0.22), and "on" (0.12) with varying weights based on their attention scores.

Simplified Example Using the Task of Translating a Sentence from English to French

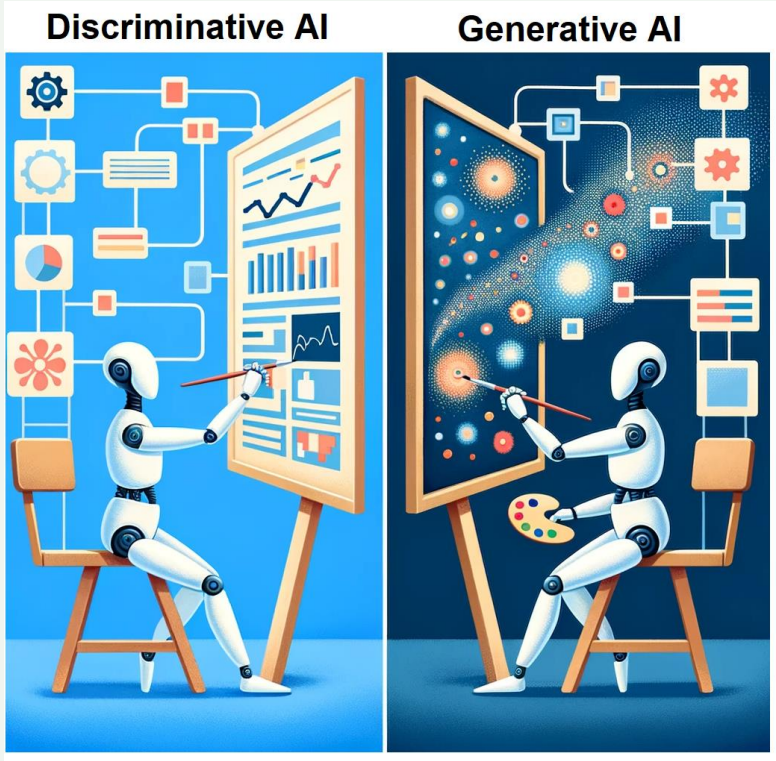
- **Step 6: Output Generation**
  - Context Vector to Output:
    - Used to generate model's output based on surrounding words.
  - Simplified Example:
    - Neural networks (complex) process the vector (in reality).
    - Here, for illustration, assume direct mapping to French:
      - "sat" -> "s'est assis" (sat down)
      - Example Context Vector: [0.3, 0.2, 0.4] (High-dimensional representation, not literal translation)
- Key Points:
  - Complex process in reality.
  - Context vector guides the model's output.

Flow of Operations

```
Input Words -> Embed -> Add Positional Encoding -> Generate Q, K, V
    |            |                    |                     |
    v            v                    v                     v
[The, cat, ...] [1,0,0], ... [1.1,0,0], ...  [Q_sat=[0,0,1.1], K=[...], V=[...]]
                                                          |
                                                          v
                                                Calculate Attention (Q*K)
                                                          |
                                                          v
                                              Scale and Apply Softmax
                                                          |
                                                          v
                                            Multiply by Values (V) -> Weighted Sum
                                                          |
                                                          v
                                              Generate Context Vector
                                                          |
                                                          v
                                             Pass Through NN -> Output Word
```

Generative AI vs Discriminative AI

- **Generative AI:**
  - Creates new and original content (text, images, code, etc.)
  - Learns patterns and examples to generate novel outputs
  - Example: ChatGPT, DALL-E, Microsoft Copilot, Gemini, Mistral, Cohere, Claude, LlaMa, etc.



- **Discriminative AI:**
  - Recognizes and classifies existing data
  - Learns to distinguish between different categories
  - Example: Identifying handwritten digits in an image.
- **Key Difference:**
  - Generative AI focuses on creating something new
  - Discriminative AI focuses on understanding existing data.

Image Source: DALL-E, OpenAI's image generation model

GPT-4 and ChatGPT

- GPT stands for **generative pre-trained transformer**.
- GPT-4 is the fourth iteration of **generative pretrained transformers** developed by OpenAI, which produce human-like text.
- **ChatGPT (Chat Generative Pre-Trained Transformer)** is built on the structure of GPT-4.
  - it is a large language model (LLM) that checks for the probability of what words might come next in sequence.
  - Uses a type of **transformer model**, in which **a neural network learns context about any language pattern** (including a spoken language or a computer programming language).
  - Also trained using **reinforcement learning from human feedback** (RLHF).

Sample Training Set

1. Context: "JavaScript is a powerful“ Target: "programming"
2. Context: "JavaScript is a versatile“ Target: "language"
3. Context: "JavaScript is a widely“ Target: "used"
4. Context: "JavaScript is a key“ Target: "technology"
5. Context: "JavaScript is a popular“ Target: "choice"
6. Context: "JavaScript is a crucial“ Target: "component"
7. Context: "JavaScript is a fundamental“ Target: "building"
8. Context: "JavaScript is a widely“ Target: "adopted"
9. Context: "JavaScript is a common“ Target: "language"
10. Context: "JavaScript is a core“ Target: "element"
11. Context: "JavaScript is a leading“ Target: "technology"
12. Context: "JavaScript is a dynamic“ Target: "language"
13. Context: "JavaScript is a necessary" Target: "tool"

14. Context: "JavaScript is a prominent" Target: "technology"
15. Context: "JavaScript is a vital" Target: "part"
16. Context: "JavaScript is a dynamic" Target: "language"
17. Context: "JavaScript is a popular" Target: "choice"
18. Context: "JavaScript is a widely" Target: "used"
19. Context: "JavaScript is a versatile" Target: "tool"
20. Context: "JavaScript is a key" Target: "element"

ChatGPT – Steps

- **Tokenization**: breaking down the sentence into individual words
  - Input: "JavaScript is"
  - Tokenized: ["JavaScript", "is"]
- **Embedding**:
  - Represent each token as an embedding vector. For example:
  - "JavaScript" -> [0.2, 0.5, -0.1]
  - "is" -> [0.8, -0.3, 0.6]
- **Positional Encoding**:
  - Add positional encodings to the embeddings:
  - "JavaScript" -> [0.2, 0.5, -0.1, 0.3, 0.1, -0.2]
  - "is" -> [0.8, -0.3, 0.6, 0.2, 0.4, -0.1]

ChatGPT – Steps

- **Self-Attention Mechanism**:
  - Calculate attention scores for each token based on their relationships:
  - Attention scores for "JavaScript": [0.6, 0.3]
  - Attention scores for "is": [0.2, 0.8]
- **Multi-Head Attention**:
  - Combine results from multiple attention heads. Suppose we have two heads:
  - Head 1: [0.7, 0.4]
  - Head 2: [0.5, 0.6]
  - Combined: [0.6, 0.3, 0.7, 0.4]

ChatGPT – Processing

- **Feedforward Neural Network**:
  - Pass the combined information through a neural network:
  - Input: [0.6, 0.3, 0.7, 0.4]
  - Output: [0.1, 0.9]
- **Layer Normalization**:
  - Apply layer normalization to stabilize the output:
  - Normalized Output: [0.3, 0.7]
- **Prediction**:
  - Predict the next word. In a simplified example, let's say the model predicts "powerful.“
  - This involves a complex probability distribution over all the possible words in the LLM's vocabulary. "Powerful" might have the highest probability in this simplified example.
- **Output**:
  - The output is the continuation of the input: "JavaScript is powerful."

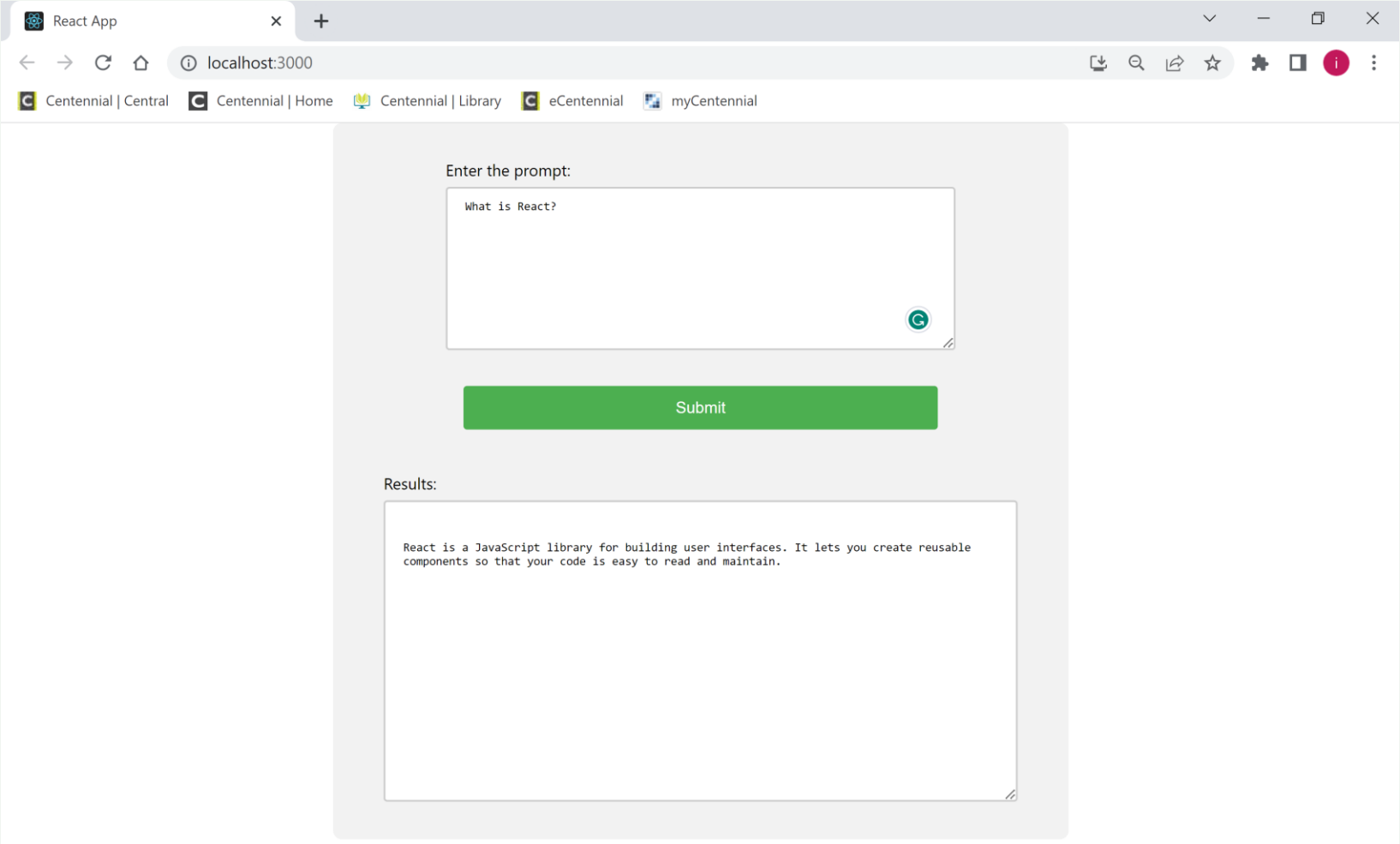
GPT React Example

```
const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
//
const { Configuration, OpenAIApi } = require("openai");
//
// Set the 'OPENAI_API_KEY  ' variable
process.env.OPENAI_API_KEY = process.env.OPENAI_API_KEY || 'my API key';
//
const configuration = new Configuration({
  apiKey: process.env.OPENAI_API_KEY
});
const openai = new OpenAIApi(configuration);
// Set up the server
const app = express();
app.use(bodyParser.json());
app.use(cors())
```

GPT React Example

```
// Set up the ChatGPT endpoint
app.post("/chat", async (req, res) => {
  // Get the prompt from the request
  const { prompt } = req.body;
  console.log('prompt=',prompt)
  // Generate a response with ChatGPT
  const completion = await openai.createCompletion({
    model: "text-davinci-003", prompt: prompt, temperature: 0, max_tokens: 60,
    top_p: 1.0,
    frequency_penalty: 0.5,
    presence_penalty: 0.0,
    stop: ["You:"],
  });
  res.send(completion.data.choices[0].text);
});
// Start the server
const port = 5000;
app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

GPT React Example



Generate Text with Gemini API (Node.js)

- This example implements the interaction with Gemini to create text.

- Key steps:
  1. Load your **API key** securely (not shown).
  2. Set **generation options** (stop words, max length, randomness).
  3. Create a **model** for text generation ("gemini-pro").
  4. Provide a **prompt** (question or instruction).
  5. **Generate** and print the response as text.

Generate Text with Gemini API (Node.js)

File Edit Selection View ··· GeminiAPI-Example

EXPLORER

OPEN EDITORS
- gemini-example.js

GEMINIAPI-EXAMPLE
- node_modules
- .env
- gemini-example.js
- package-lock.json
- package.json

OUTLINE

TIMELINE

gemini-example.js > generationConfig

```javascript
require('dotenv').config();
//
// Import the GoogleGenerativeAI class
const { GoogleGenerativeAI } = require("@google/generative-ai");
// Access your API key as an environment variable (see "Set up your API
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

// Define the generation configuration
const generationConfig = {
  // The model will stop generating content
  // after it reaches a stop sequence
  stopSequences: ["red"],
  // the maximum number of tokens that can be generated in the response
  maxOutputTokens: 200,
  // The temperature controls the degree of randomness in token selecti
  // Lower temperatures are good for prompts that require a more determ
  // or less open-ended response
  temperature: 0.9,
  // The topP parameter changes how the model selects tokens for output
  topP: 0.1,
  // The topP parameter changes how the model selects tokens for output
  // Tokens are selected from the most to least probable until the sum
  // their probabilities equals the topP value
  topK: 16,
};
// Create a generative model
const model = genAI.getGenerativeModel({ model: "MODEL_NAME",  generati
```

⊗ 0 ⚠ 0 0 | Ln 9, Col 27 | Spaces: 2 | UTF-8 | CRLF | {} JavaScript

Prompt Engineering: Guiding AI Conversations

- Prompt engineering is the **art of crafting instructions for large language models (LLMs)** to achieve desired results.
  1. **What it is**: The process of designing prompts (text instructions) that effectively communicate with LLMs for specific tasks.
  2. **Why it matters**: Precise prompts help LLMs understand what you want and generate better outputs (text, code, etc.).
  3. **Key elements**: Clear instructions, relevant context, desired style or tone, and controlling randomness.
  4. **Benefits**: Unlocks LLM capabilities, improves accuracy and relevance of responses, reduces ambiguity.
  5. **Applications**: Widespread! From writing different creative text formats to translating languages or generating code.
  6. **It's a skill**: Mastering prompt engineering requires practice and understanding LLM strengths and limitations.

Types of Prompts

- **Instructional** prompts: Clearly state the task and desired outcome (e.g., "Write a GraphQL query to fetch all blog posts with their titles and authors from a sample API").
- **Open ended** prompts: Spark creativity with a starting point or theme (e.g., "Create a single-page weather application using a modern web framework (e.g., Svelte) that displays real-time weather data.").
- **Continuational** prompts: Build upon existing content, adding a new direction or element (e.g., "You have a GraphQL query that retrieves basic product data. Now, modify it to include reviews and average rating for each product.").
- **Contrastive** prompts: Compare and contrast topics, highlighting similarities or differences (e.g., "Write a REST API endpoint to retrieve all users from a database. Now, rewrite the functionality using a GraphQL query.").
- **Informative** prompts: Provide background knowledge or context to guide the LLM (e.g., "Given the Vue.js framework, write code for a reusable component that displays a product card with image, title, and price.").

Prompt Engineering Techniques

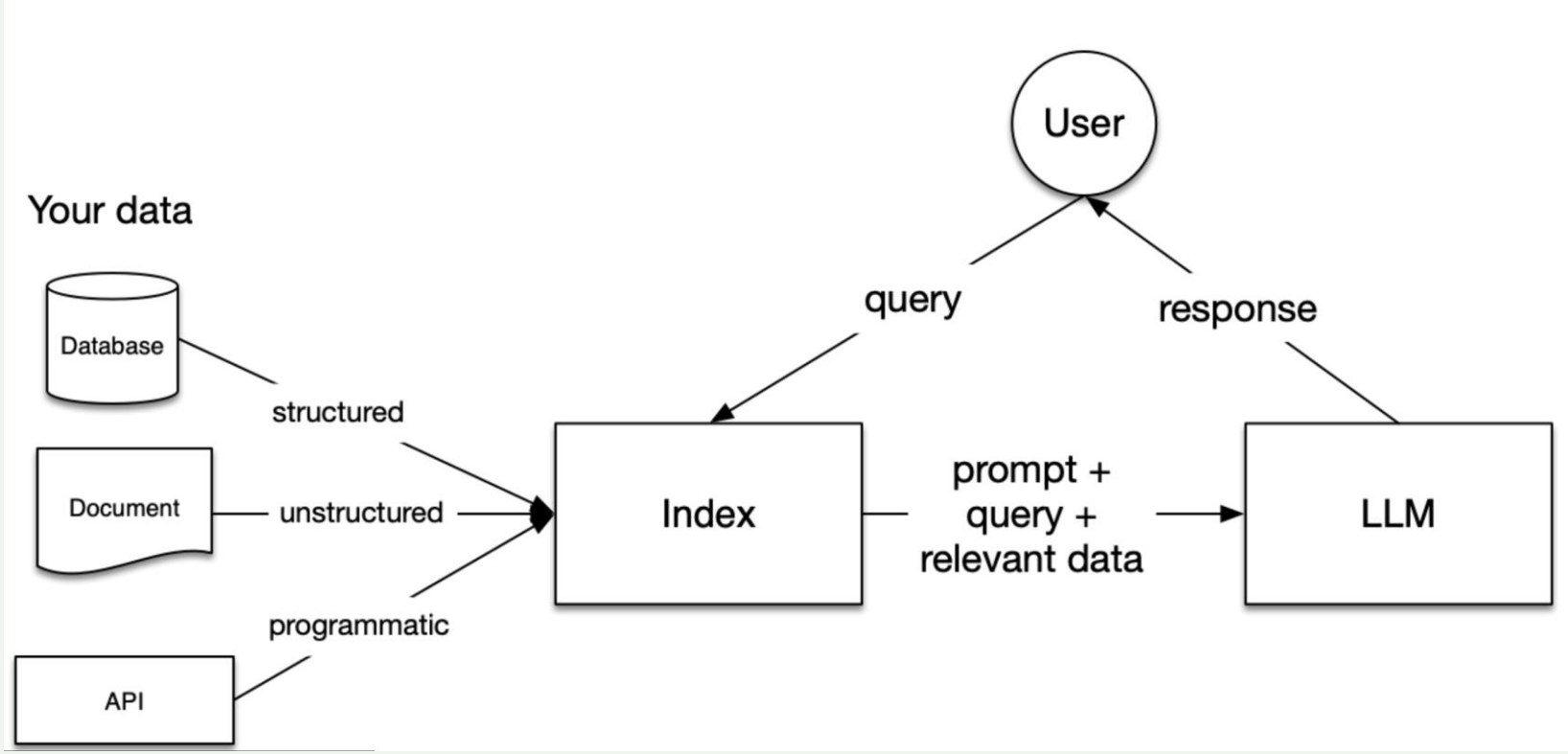
- **Zero-Shot Prompting**: Clear instructions without training data. (e.g., "Write a function to check if a year is a leap year.")
- **Few-Shot Prompting**: Provide a few examples to guide the LLM. (e.g., "Write a function to count vowels in a string, like these functions for length and uppercase conversion.")
- **Chain-of-Thought Prompting** (CoT): Break down tasks into logical steps. (e.g., "Calculate average: Initialize total, loop through numbers, add to total, divide by number of elements.")
- **Tree-of-Thought Prompting** (ToT): Explore multiple paths within a complex task. (e.g., "Analyze code for errors, check for division by zero and out-of-bounds array access.")
- **Contrastive Prompting**: Use positive and negative examples to guide the LLM.
- **Iterative Prompting**: Refine the prompt based on the LLM's initial outputs.
- **Meta-Prompting**: Train a smaller model to automatically generate prompts for a larger LLM.

Retrieval Augmented Generation (RAG)

- RAG combines retrieval-based and generative approaches to enhance the capabilities of Large Language Models (LLMs).
  - **Information Retrieval**: An initial retrieval step searches for relevant information from a data source (like documents, code, or knowledge graphs) based on the user's prompt or query.
  - **Augmentation**: The retrieved information is presented to the LLM alongside the original prompt.
  - **Generation**: The LLM leverages both the prompt and the retrieved data to generate a more informed and comprehensive response.
- Benefits include:
  - **Improved Accuracy**: Access to relevant context ensures the LLM's response aligns with factual information.
  - **Enhanced Creativity**: The retrieved data can spark new ideas and lead to more creative and interesting generations.
  - **Reduced Hallucination**: LLMs are less likely to fabricate information when supplemented with real-world data.

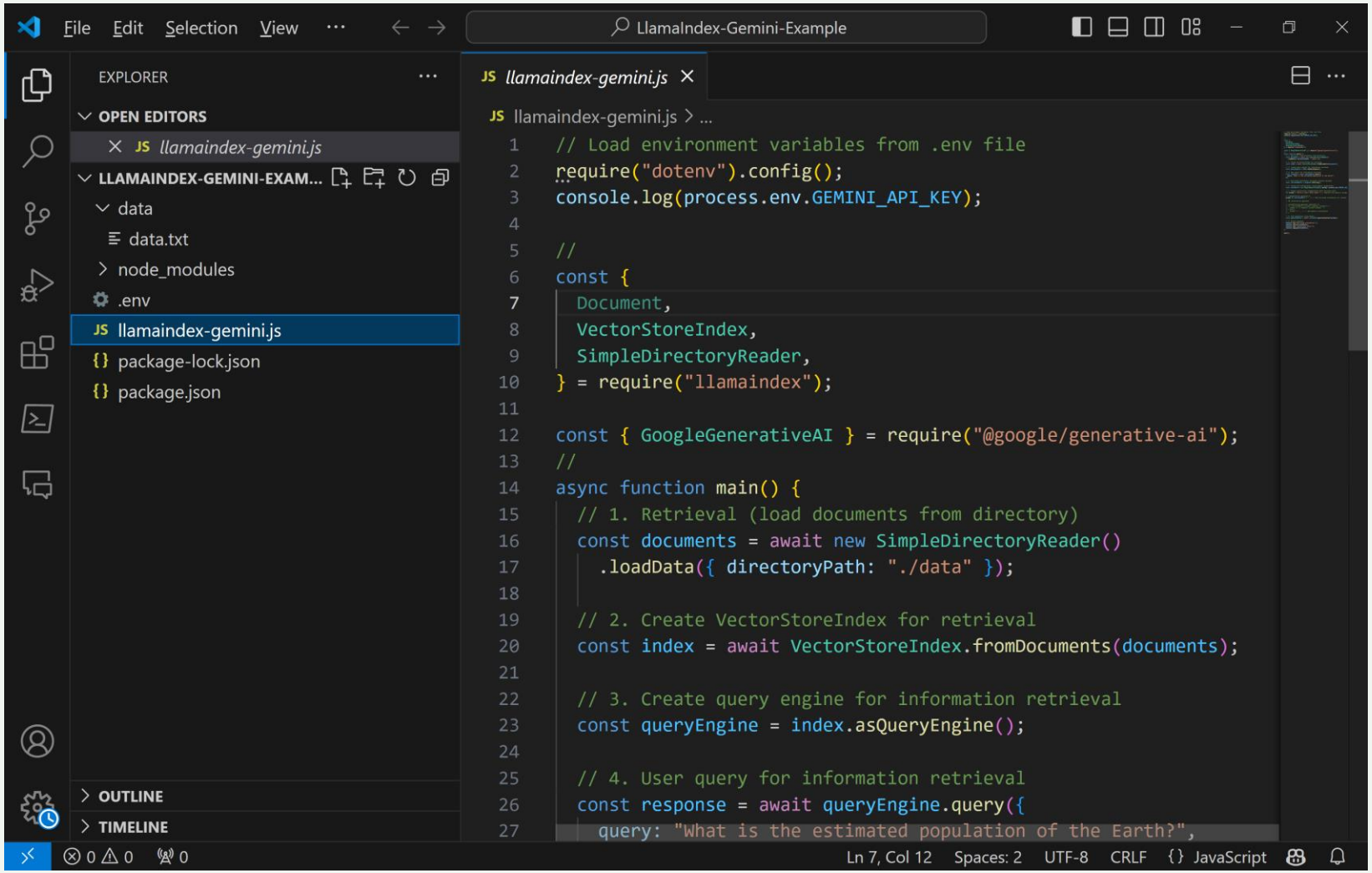
RAG Architecture with LlamaIndex

- **LlamaIndex** is a simple, flexible data framework for connecting custom data sources to large language models (LLMs).



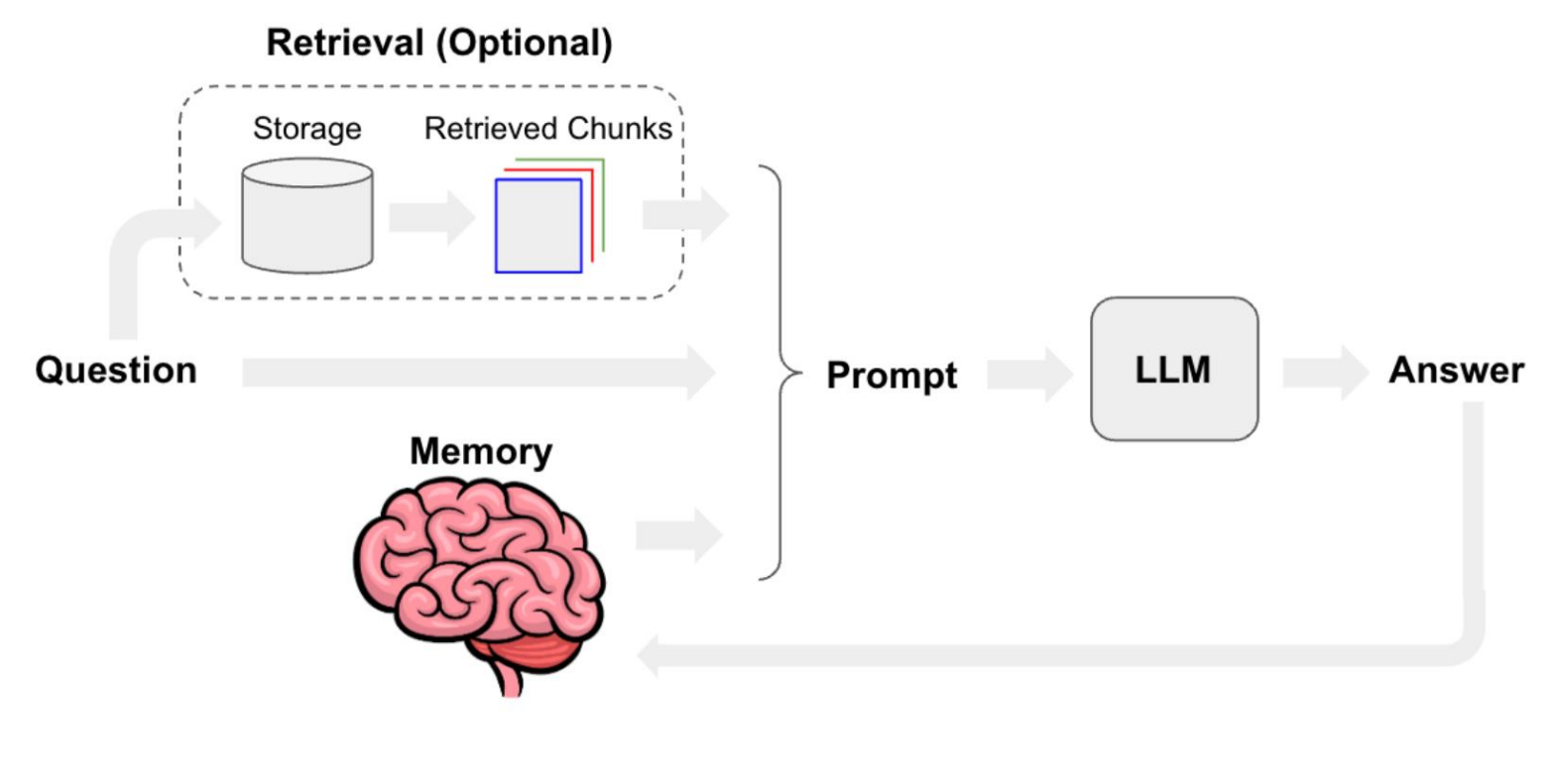
(https://docs.llamaindex.ai/en/stable/getting_started/concepts/)

Retrieval Augmented Generation (RAG) with LlamaIndex



RAG Architecture with LangChain

- **LangChain** is a framework for developing applications powered by large language models (LLMs).
  - Question answering with RAG
  - Extracting structured output
  - Chatbots



https://js.langchain.com/docs/use_cases/chatbots/

RAG Architecture with LangChain

```
File Edit Selection View ···   langchainjs-gemini-example

EXPLORER
OPEN EDITORS
  index.js
LANGCHAINJS-GEMINI-EXAMPLE
  data
    data.pdf
    info.txt
  node_modules
  .env
  index.js
  index1.js
  package-lock.json
  package.json
OUTLINE
TIMELINE

index.js > model
55  async function generateResponse(query) {
70      return responses;
71    } catch (error) {
72      console.error("Error during data retrieval or generation:", error);
73      return "An error occurred while generating the response.";
74    }
75  }
76
77  generateResponse("What is the estimated population of the Earth?")
78    .then(response => console.log("Generated Response:", response.content))
79    .catch(error => console.error(error));
80

OUTPUT  TERMINAL  PORTS
TERMINAL                                  powershell
Processing sentence: The population of the earth is estimated to be over 8 billion peo
ple. That's a lot of people sharing our planet!
Processing sentence: Honeybees can visit up to 100 flowers per hour, collecting nectar
 and pollen to make honey. These busy insects are essential for pollinating plants, wh
ich helps them reproduce.
Processing sentence: The world's oldest known tree is a bristlecone pine named Methuse
lah, located in California. It's estimated to be over 4,800 years old!Processing sente
nce: The human brain is made up of about 86 billion neurons, which are connected by tr
illions of synapses. This complex network allows us to think, learn, and feel.
Generated Response: Over 8 billion
PS C:\Classes\COMP308\Examples\2024\langchainjs-gemini-example>

⊗ 0 ⚠ 0   0   Ln 10, Col 25   Spaces: 2   UTF-8   CRLF   {} JavaScript
```

Ollama - Running large language models locally

- Ollama is a tool that allows you to run open-source large language models (LLMs) locally on your machine.
- Download it here: https://github.com/ollama/ollama
- To run and chat with Llama 2:

ollama run llama2

- First time will install llama2.

Command Prompt

```
C:\Users\inika>ollama run llama2
>>> what is ollama?

I believe you may be thinking of "alpaca." Alpacas are domesticated animals that are native to South America and
are known for their soft, warm fleece and gentle nature. They are often used for their wool and are considered a
delicacy in some cultures.

>>> could you draw a llama using ascii characters?

Of course! Here is a simple drawing of a llama using ASCII characters:
```
 /_/\
(o.o)
 > ^ <
```

>>> /bye

C:\Users\inika>
```

Ollama LangChain RAG Example

File Edit Selection View ··· ollama-langchain-js-example

EXPLORER

OPEN EDITORS
- ollama-rag.js

OLLAMA-LANGCHAIN-JS-EXAMPLE
- data
- node_modules
- ollama-chatbot.js
- ollama-example.js
- ollama-rag.js
- ollama-sentiment.js
- package-lock.json
- package.json
- readme.txt

OUTLINE
TIMELINE

ollama-rag.js > ...

```
34  async function generateResponse(query) {
36      const retrievedData = await retrieveData(query);
37      const augmentedQuery = retrievedData ? `${query} Considering the followi
38
39      // Send the augmented query to Ollama and get a response
40      const response = await ollama.invoke(augmentedQuery);
41      return response;
42    } catch (error) {
43      console.error("Error during data retrieval or generation:", error);
44      return "An error occurred while generating the response.";
45    }
46  }
```

OUTPUT TERMINAL PORTS

TERMINAL powershell

```
PS C:\Classes\COMP308\Examples\2024\ollama-langchain-js-example> node ollama-rag
Generated Response:
The estimated population of the Earth as of 2023 is approximately 7.9 billion people
. This number is constantly changing due to factors such as births, deaths, and migr
ation. According to the United Nations Department of Economic and Social Affairs Pop
ulation Division, the world's population is projected to reach around 9.7 billion by
 2050 and 11.2 billion by 2100. However, it's important to note that population proj
ections are subject to uncertainty and can be influenced by a variety of factors, in
cluding changes in fertility rates, mortality rates, and migration patterns.
PS C:\Classes\COMP308\Examples\2024\ollama-langchain-js-example>
```

⊗ 0 ⚠ 0 0 Ln 1, Col 1 Spaces: 2 UTF-8 CRLF {} JavaScript

References

- Attention Is All You Need: https://arxiv.org/abs/1706.03762
- https://jalammar.github.io/illustrated-transformer/
- https://ai.google.dev/tutorials/get_started_node
- https://platform.openai.com/docs/quickstart?context=node
- https://ts.llamaindex.ai/
- https://docs.llamaindex.ai/en/stable/getting_started/concepts/
- https://survey.stackoverflow.co/2023/
- https://openai.com/blog/planning-for-agi-and-beyond
- https://karpathy.medium.com/software-2-0-a64152b37c35
- https://github.com/langchain-ai/langchain
- https://github.com/ollama/ollama
- https://ollama.com/library/llama2
- https://arxiv.org/abs/2403.08299